

CAREERCASE × SIH26044 | PR1 PRE-IMPLEMENTATION INSPECTION REPORT

Owner: Harsh **Branch:** feature/harsh/opportunity-authoring-recruiter-pr1

Base: integration/sih26044-product-v0.2 **Generated:** 2026-08-29

Status: Inspection only — zero files modified

✓ SAFE TO START WITHIN CURRENT CONTRACTS

Owner: Harsh

Branch: feature/harsh/opportunity-authoring-recruiter-pr1

Base: integration/sih26044-product-v0.2

Date: 2026-08-29

Status: Inspection only — zero files modified

1. Task Understanding

What PR1 must deliver (plain language)

There are **two main surfaces**, both entirely on the industry/recruiter side:

Surface A — Opportunity Authoring

An employer-facing multi-section draft form that lets a recruiter (Riya-persona) build and publish a structured job/internship opportunity. Every requirement must:

- Preserve the exact literal wording typed by the author
- Track whether a skill was canonically resolved, needs human review, or is unresolved
- Carry a `HumanConfirmationTrace` before the opportunity version may be called "confirmed"
- Support an optional AI-assisted extraction preview — with source text shown, confidence disclosed, and accept/edit/reject actions required — but **AI never auto-publishes**
- Produce a versioned `OpportunityVersion` (type, audiences, requirements, eligibilityRules, source) that Laya's student explorer can consume by `opportunityId`

Surface B — Recruiter Applicant Workspace

An industry-facing list of applications for opportunities the recruiter's organization owns. From that list:

- Open an applicant detail view backed **exclusively** by the `RecruiterApplicationProjection` / `ProductionRecruiterProjection` allowlist
- Display readiness band as decision support (not as a hiring probability or ranking)

- Record explicit human stage actions (`under_review` → `shortlisted` → `offered` / `rejected_by_human`) with notes, using `SihBrowserDal.transitionApplicationStage()`
- Block detail access immediately when consent is withdrawn — never cache a full applicant object past the consent check

What is explicitly out of scope

OUT OF SCOPE	WHY
Automatic shortlisting, ranking, scoring, rejection	Non-Negotiable Rule 3, 4, 5
RIASEC, values, aspirations, counselor data in any view	Rules 6, 11; <code>ProhibitedRecruiterData</code> contract
ATS / NCS / AICTE / employer live integrations	Not implemented; must stay labelled as controlled prototype
Institution/faculty workflows	Nipun's ownership
Student-side gap-closure, readiness calculation	Laya + Madhu ownership
Evidence/verification detail outside snapshot	Manvil's ownership
Engine B changes	Kamal-only
Schema migrations	Kamal-only; all 18 existing migrations are frozen for Harsh
Broad talent search across candidates	Rules 3, 9
Changing production projection allowlist	Rule 13

2. Existing Implementation to Reuse

Domain types (`src/app/domain/`) — read-only, never edit

FILE	WHAT HARSH USES
<code>opportunity.ts</code>	<code>Opportunity</code> , <code>OpportunityVersion</code> , <code>OpportunityType</code> , <code>OpportunityAudience</code> , <code>OpportunityRequirement</code> (all 7 union variants), <code>RequirementPriority</code> , <code>RequirementImportance</code> , <code>RequirementEvidenceExpectation</code> , <code>EligibilityRule</code> , <code>EligibilityRuleDefinition</code>
<code>skillResolution.ts</code>	<code>CanonicalResolutionState</code> (3 states: <code>resolved</code> , <code>review_required</code> , <code>unresolved</code>), <code>SkillReviewSuggestion</code>
<code>application.ts</code>	<code>Application</code> , <code>ApplicationSnapshot</code> , <code>ApplicationEvent</code> , <code>ApplicationTransitionEvent</code> , <code>HumanRejectionApplicationEvent</code> , <code>APPLICATION_STAGES</code> , <code>ApplicationStage</code>
<code>consent.ts</code>	<code>ConsentRecord</code> , <code>ConsentPurpose</code> , <code>ProhibitedRecruiterData</code> , <code>AuditEvent</code>
<code>readiness.ts</code>	<code>ReadinessBand</code> , <code>RequirementState</code> , <code>EligibilityStatus</code> , <code>OpportunityReadinessResult</code> , <code>RequirementReadinessResult</code> (display-only)
<code>evidence.ts</code>	<code>EvidenceProvenance</code> , <code>VerificationState</code> , <code>EvidenceRecord</code> (within snapshot context only)
<code>shared.ts</code>	<code>HumanConfirmationTrace</code> , <code>HumanConfirmationMethod</code> , <code>VersionStamp</code> , <code>SourceReference</code> , branded ID types
<code>identity.ts</code>	<code>Organization</code> , <code>OrganizationMembership</code> , <code>ActorRole</code> (<code>'recruiter'</code>), <code>ActingContext</code>

Services to reuse

FILE	METHODS HARSH MAY CALL
<code>SihBrowserDal</code>	<code>transitionApplicationStage()</code> (stage events), <code>grantConsent()</code> / <code>withdrawConsent()</code> (read consent status), <code>createApplication()</code> (if needed from recruiter side — likely not)
<code>productionRecruiterProjection.ts</code>	<code>buildProductionRecruiterProjection()</code> , <code>validateNoProhibitedKeys()</code> , <code>ProductionRecruiterProjection</code> type
<code>recruiterProjection.ts</code>	<code>RecruiterApplicationProjection</code> type (used in demo state; production uses the <code>productionRecruiterProjection</code> version)

Demo runtime to read but NOT modify

FILE	WHAT IT SHOWS
<code>demoFixtures.ts</code>	<code>DEMO_REQUIREMENTS</code> (4 requirements showing all <code>CanonicalResolutionState</code> variants), <code>DEMO_ORGANIZATIONS</code> , <code>DEMO_PERSONAS.recruiter</code> (<code>Riya</code>)
<code>demoReducer.ts</code>	Reference for how <code>START_HUMAN_REVIEW</code> , <code>SHORTLIST_APPLICATION</code> , <code>RECORD_SELECTED_OUTCOME</code> actions work; Harsh must mirror this pattern
<code>DemoPages.tsx</code> → <code>DemoRecruiterPage</code>	Shows the exact allowlisted fields the recruiter may see; Harsh's real recruiter workspace must honour the same field set

QA scripts (run, not edit)

All 6 required scripts already exist in `scripts/` :

- `npm run qa:recruiter-projection` → `scripts/recruiter-projection-qa.ts`
- `npm run qa:opportunity-readiness` → `scripts/opportunity-readiness-qa.ts`
- `npm run qa:sih-boundary` → `scripts/sih-boundary-qa.ts`
- `npm run qa:demo-flow` → `scripts/demo-flow-qa.ts`
- `npm run qa:evidence-integrity` → `scripts/evidence-integrity-qa.ts`
- `npm run typecheck` / `npm run build`

Key architectural facts discovered

- No `sih/industry/` or `sih/recruiter/` directory exists yet — both must be created fresh. This is clean; there is no collision risk.
- `Routes.ts` has no `/industry/` or `/recruiter/` routes — these will need to be added. This is a **protected file** (see §4) — the minimal registration needs Kamal's review. See §9 for the resolution path.
- The demo runtime already has a working recruiter page at `/demo/recruiter` using `RecruiterApplicationProjection` — Harsh's real pages replicate the same field boundaries.
- `buildProductionRecruiterProjection()` has a built-in `validateNoProhibitedKeys()` that throws at runtime if any prohibited key is passed — this is the enforcement layer.

- `SihBrowserDal.transitionApplicationStage()` is the only browser-callable method for recording human application stage events — it writes to `application_events` via RLS.
- **No opportunity authoring DAL method exists in `SihBrowserDal`** — opportunity insert/update appears to be a trusted Worker operation or the recruiter writes to a browser-RLS-permitted table. This must be confirmed before building the authoring form (see §9, Risk R1).

3. Proposed Files

Files to CREATE (all new, under Harsh's owned directories)

<code>src/app/components/sih/industry/</code>	
<code>OpportunityAuthoringShell.tsx</code>	Multi-section draft shell (H2)
<code>OpportunityBasicsSection.tsx</code>	Title, type, audience, description (H2)
<code>OpportunityRequirementsSection.tsx</code>	Per-requirement form with resolution badges (H2)
<code>OpportunityEligibilitySection.tsx</code>	Eligibility rule builder (H2)
<code>OpportunityLogisticsSection.tsx</code>	Dates, location, capacity (H2)
<code>OpportunityQuestionnaireSection.tsx</code>	Employer questionnaire items (H2)
<code>OpportunityReviewPanel.tsx</code>	Pre-publish summary + human confirm (H3)
<code>SkillResolutionBadge.tsx</code>	resolved / review_required / unresolved indicator (H2)
<code>RequirementReviewCard.tsx</code>	Literal source + AI suggestion + accept/edit/reject (H3)
<code>OpportunityPublishConfirmDialog.tsx</code>	Final human-confirm gate before publish action (H3)
<code>src/app/components/sih/recruiter/</code>	
<code>RecruiterWorkspaceShell.tsx</code>	Layout + opportunity filter shell (H4)
<code>ApplicationListTable.tsx</code>	Stage/date ordered list, no readiness rank (H4)
<code>ApplicationDetailView.tsx</code>	Projection-only applicant detail (H4)
<code>ReadinessBandExplanation.tsx</code>	Display band + requirement states, not probability (H4)
<code>HumanStageActionPanel.tsx</code>	review / shortlist / offer / reject_by_human actions (H5)
<code>ApplicationEventTimeline.tsx</code>	Append-only human event history (H5)
<code>ConsentWithdrawalGuard.tsx</code>	Blocks detail when consent status ≠ 'granted' (H5)

Why each file is needed: each directly maps to a section or user action in the Task Execution Plan milestones H2–H5. No file duplicates an existing component or domain type.

Files to MODIFY (minimal, surgical)

FILE	CHANGE NEEDED	RISK
<code>src/app/routes.ts</code>	Register <code>/industry/opportunities/</code> and <code>/recruiter/applications/</code> route entries	Protected — needs Kamal approval first (see §4, §9)

Files intentionally NOT touched

Everything listed in §4 below.

4. Protected / Shared Files — Will NOT Modify Without Kamal Approval

FILE / DIRECTORY	REASON FROZEN
<code>src/app/domain/**</code>	Shared domain contracts; any change must follow the shared-change request format
<code>src/app/engine/**</code>	Engine B (Opportunity Readiness + all sub-engines); Kamal-only
<code>src/app/services/sih/**</code>	Shared DAL, Trusted API client, production projection; Kamal-owned
<code>src/app/routes.ts</code>	Shared routing; route registration for new prefixes requires Kamal patch
<code>src/app/App.tsx</code>	Root layout
<code>src/app/demo/DemoLayout.tsx</code>	Shared demo chrome
<code>src/app/demo/DemoSihRuntime.tsx</code>	Demo runtime wrapper
<code>src/app/demo/demoTypes.ts</code>	Shared demo state contract
<code>src/app/demo/demoFixtures.ts</code>	Shared synthetic data
<code>src/app/demo/demoReducer.ts</code>	Shared demo state machine
<code>src/app/demo/demoScenario.ts</code>	Shared derived-state helpers
<code>supabase/migrations/**</code>	All 18 migrations frozen
<code>supabase/tests/**</code>	Supabase test suite
<code>worker/**</code>	Cloudflare Worker / Engine B server side
Global styles / generic UI primitives	Shared across all owners
Any file under <code>src/app/components/sih/student/</code> , <code>sih/evidence/</code> , <code>sih/faculty/</code> , <code>sih/verification/</code>	Owned by Laya/Madhu, Manvil, Nipun respectively

5. Data and Authority Mapping

USER ACTION	DATA SOURCE	WRITE AUTHORITY	PRIVACY CONSTRAINT
View opportunity authoring form	Local component state (draft)	Browser — no DB write until publish	n/a
Skill resolution lookup	<code>domain/skillResolver.ts</code> (in-memory canonical skill index)	Read-only; no DB write	n/a
AI extraction preview	Controlled fixture / replay (no live AI)	Suggestion only; never auto-applied	Must show source text beside suggestion
Human confirm requirement	UI state → sets <code>humanConfirmed: true</code> , <code>confirmationMethod</code>	Local draft state; saved on publish	<code>HumanConfirmationTrace</code> required
Publish opportunity (create version)	Must go through authorized write path	Unknown — see Risk R1 below	Opportunity must carry <code>ownerOrganizationId</code> matching actor's org
List applications for owned opportunity	Read <code>applications</code> via Supabase RLS (<code>can_recruiter_read_application</code>)	Read-only	Only applications for recruiter's org's opportunities
Open applicant detail	<code>buildProductionRecruiterProjection()</code> from snapshot + events	Read-only	<code>validateNoProhibitedKeys()</code> blocks prohibited fields; consent must be <code>'granted'</code>
Record human stage action	<code>SihBrowserDal.transitionApplicationStage()</code>	RLS-permitted INSERT into <code>application_events</code>	<code>toStage:</code> <code>'rejected_by_human'</code> requires <code>reason</code> ; <code>eventKind</code> auto-set
Consent withdrawal check	Read <code>consent_grants</code> + <code>consent_lifecycle_events</code>	No write from recruiter side	If <code>ConsentRecord.status !== 'granted'</code> → block detail, show disclosure
View readiness band	From <code>RecruiterApplicationProjection.readinessBand</code> (snapshot-frozen)	Read-only	Label as "readiness at submission", never as "hiring probability"

6. Implementation Milestones

MILESTONE	MAPPED TASK PLAN SECTION	VERTICAL SLICE
H1 — Inspect	This report	✓ Complete
H2 — Structured authoring	Step 3 of guide	Create <code>OpportunityAuthoringShell</code> → <code>BasicsSection</code> → <code>RequirementsSection</code> with resolution badges → <code>EligibilitySection</code> → <code>LogisticsSection</code> → <code>QuestionnaireSection</code> . Each section independently previewable in the demo route. Commit: <code>feat(industry): add structured opportunity authoring</code>
H3 — Assistive normalization + publish	Step 4 of guide	Add <code>RequirementReviewCard</code> (source + suggestion + accept/edit/reject), <code>OpportunityPublishConfirmDialog</code> (human confirm gate). Replay/offline label in UI. Commit: <code>feat(industry): add human-confirmed requirement review</code>
H4 — Recruiter workspace listing	Step 5 of guide	<code>RecruiterWorkspaceShell</code> + <code>ApplicationListTable</code> (stage/date ordered). Connect to <code>ProductionRecruiterProjection</code> . Consent-withdrawal guard wired. Commit: <code>feat(recruiter): add consent-bound applicant review</code>
H5 — Human recruitment actions	Step 5 (continued)	<code>HumanStageActionPanel</code> + <code>ApplicationEventTimeline</code> . Test: <code>rejected_by_human</code> requires reason; auto-reject path does not exist. Test consent withdrawal before happy path.
H6 — Verify + PR	Step 6 + Step 7	Run all 6 QA commands, <code>npm run build</code> . Attach screenshots. Draft PR with exact test output and manual checklist.

Slice rule: each milestone is locally previewable in the demo runtime at `/demo/recruiter` and in Harsh's new route shell before moving to the next.

7. Testing Plan

Automated QA (exact commands from guide)

```
npm run typecheck
npm run qa:recruiter-projection
npm run qa:opportunity-readiness
npm run qa:sih-boundary
npm run qa:demo-flow
npm run qa:evidence-integrity
npm run build
```

What each command checks relevant to Harsh:

- `qa:recruiter-projection` — verifies that the projection source file contains no `CareerPassport`, `GuidanceContext`, Engine A imports, or generic student-object spread; verifies that every evidence item is covered by application_review consent; verifies the exact allowlisted top-level field set
- `qa:sih-boundary` — checks Engine B files for private guidance leakage (`RIASEC`, `aspirations`, etc.), readiness-percentage framing, and connector truth violations

- `qa:opportunity-readiness` — exercises the canonical `computeOpportunityReadiness()` with controlled inputs; Harsh's new requirements must pass this unchanged
- `qa:demo-flow` — runs the full reducer sequence through submit → recruiter review; must not break after Harsh's new demo-adjacent code
- `qa:evidence-integrity` — verifies append-only evidence provenance rules

Manual verification checklist

STATE TO VERIFY	EXPECTED BEHAVIOUR
Authoring form — unresolved skill	Shows literal text + <code>unresolved</code> badge; never substitutes a canonical skill
Authoring form — <code>review_required</code> skill	Shows suggestions with <code>reviewOnly: true</code> ; accept/edit/reject required before confirm
Publish button	Disabled until every requirement has <code>humanConfirmed: true</code>
AI suggestion disclosure	Source text visible beside suggestion; "controlled prototype" label present
Recruiter workspace — no applications	Empty state: "No applications received" with explanation
Recruiter workspace — consent withdrawn	Detail blocked; reason disclosed: "Applicant has withdrawn consent"
Recruiter workspace — wrong org	403-style: "You do not have access to this opportunity's applications"
Applicant detail readiness band	Labelled "Readiness at submission — not a hiring probability"
<code>rejected_by_human</code> action	Requires reason field; disabled if reason is empty
Stage action history	Every event shows actor + timestamp + note; no auto-system events
Mobile ($\leq 375\text{px}$)	All panels stack; action buttons min-height $\geq 44\text{px}$; visible focus rings
Keyboard-only	Tab order logical; all actions reachable without mouse
No real applicant data	All displayed data is labelled "Controlled demo fixture"

8. Dependencies on Other Teammates

What Harsh receives from teammates

TEAMMATE	WHAT HARSH RECEIVES	HOW
Madhu	<code>ApplicationSnapshot</code> with <code>submittedReadiness</code> , <code>evidenceRecordIds</code> , <code>consentRecordIds</code> , <code>contentHash</code>	Read-only from DB via RLS; Madhu owns the creation path
Madhu	<code>ApplicationEvent</code> records (student-side <code>applying</code> → <code>applied</code> transition)	Read-only; Harsh only appends recruiter-side events
Manvil	<code>VerificationEvent</code> details that flow into the <code>RecruiterApplicationProjection.evidence[].verificationState</code>	Already embedded in snapshot; Harsh displays, never recreates
Laya	Navigational context: <code>opportunityId</code> + <code>opportunityVersionId</code> that student was browsing when applying	Identifier only; Harsh publishes the version, Laya consumes it

What Harsh hands off to teammates

TO	WHAT	HOW
Laya	Published <code>opportunityId</code> + <code>opportunityVersionId</code> with versioned <code>requirements</code> and <code>eligibilityRules</code>	Laya reads from DB; Harsh does not call Laya's code

Hard boundaries

- Harsh does **not** implement student gap-closure, evidence upload, or consent grant flows — those are Madhu and Manvil's surfaces.
 - Harsh does **not** implement faculty collaboration views — that is Nipun.
 - Harsh does **not** touch `buildProductionRecruiterProjection()` internals — consume it only.
-

9. Risks and Blockers

#	RISK / BLOCKER	SEVERITY	CATEGORY	RESOLUTION
R1	No opportunity authoring write path in the browser DAL. <code>SihBrowserDal</code> has no <code>insertOpportunity()</code> or <code>upsertOpportunityDraft()</code> method. The demo fixture hardcodes <code>DEMO_FIXTURE</code> . Whether a browser RLS-permitted INSERT on <code>opportunities</code> / <code>opportunity_versions</code> exists, or whether this must go through the Worker, is unknown from code inspection alone.	 Blocker for H2 publish step	Engine B / Supabase RLS	Ask Kamal: "Does the <code>sih26044</code> schema permit browser-direct INSERT on <code>opportunities</code> and <code>opportunity_versions</code> tables for a recruiter role, or must publish go through a trusted Worker RPC?"
R2	Routes.ts is a protected shared file. Adding <code>/industry/opportunities/</code> and <code>/recruiter/applications/</code> routes requires editing <code>src/app/routes.ts</code> , which is owned by Kamal.	 Medium	Shared route	Request a minimal integration-owned patch: two lazy-loaded route entries under the <code>DemoSihRuntime</code> subtree (or a new subtree) for Harsh's new page components. This can be done at H4 latest, or earlier as a stub.
R3	No <code>can_recruiter_read_application</code> query helper in the browser DAL. The teammate-boundaries doc references this RLS function, but <code>SihBrowserDal</code> exposes no method to query applications for an owned opportunity. Harsh will need to call <code>supabase.schema('sih26044').from('applications').select(...)</code> directly, relying on RLS.	 Medium	DAL gap	This is browser-permitted (RLS enforces ownership). Confirm the column shape with Kamal before writing the query; do not invent column names.
R4	AI extraction fixture does not yet exist. H3 calls for a "controlled extraction fixture" for assistive review. No such fixture is currently in <code>demoFixtures.ts</code> .	 Medium	Controlled prototype	Build H3 with a local-component-only replay fixture (hardcoded suggestions for the demo opportunity's skills). Label clearly as "Controlled prototype — not live AI". Disclose in PR.
R5	<code>sih-boundary-qa.ts</code> pattern <code>\bAccessibility\b/i</code> would match on the string "accessibility" in UI labels (e.g., <code>aria-label="Accessibility information"</code> comments). Check new component source doesn't trigger the regex on source-file scan.	 Low	QA script	Avoid the word "accessibility" as a standalone identifier in source; use <code>ally</code> or full <code>aria-*</code> attributes as values, not variable names.
R6	Consent withdrawal requires a real-time read. The UI must check <code>ConsentRecord.status</code> on every applicant detail open — not just at load. No real-time subscription is wired in the current browser DAL.	 Medium	Consent	For PR1, perform a fresh read on each detail panel open. Do not cache the consent status in component state across navigations.

Changes that require Kamal's explicit approval before implementation

→ Any new method added to `SihBrowserDal` (R1 resolution path)

- Any new Worker endpoint for opportunity authoring (R1 if the answer is "go through Worker")
- Route registration in `src/app/routes.ts` (R2)
- Any new Supabase migration for opportunity authoring columns not already in migrations 001–018 (unlikely needed — the `opportunities` and `opportunity_versions` tables already exist)

10. Final Pre-Implementation Verdict

✓ SAFE TO START WITHIN CURRENT CONTRACTS

Rationale:

- All domain types needed (Opportunity, OpportunityRequirement, CanonicalResolutionState, Application, ApplicationSnapshot, ConsentRecord, ProductionRecruiterProjection) exist and are complete. No shared-contract additions are required to begin milestones H2, H4, H5.
- All QA scripts exist and can run today against existing code.
- Harsh's component directories (`sih/industry/`, `sih/recruiter/`) are empty — no collision risk.
- The demo fixture already has an `OpportunityVersion` with all three `CanonicalResolutionState` variants, which is the primary test case for skill resolution UI.
- **Blocker R1 (authoring write path)** affects only the **publish step** in H2/H3, not the form-building or recruiter-workspace work. Harsh can safely build H2's draft form UI, H4's recruiter workspace, and H5's human stage actions while waiting for R1 clarification.
- **Route registration (R2)** can be deferred: build components first, preview locally by importing them directly into a demo page. Submit a route registration request to Kamal before H4 is complete.

Recommended first commit sequence:

- Branch: `git switch -c feature/harsh/opportunity-authoring-recruiter-pr1`
- Confirm `npm run typecheck` passes on the baseline.
- Create `src/app/components/sih/industry/` and `src/app/components/sih/recruiter/` directories.
- Open a Draft PR after the first coherent H2 slice (basics + requirements sections with resolution badges visible in local dev).
- Ask Kamal: R1 (authoring write path) and R2 (route stub) in parallel with H2 UI work.

Stop conditions: pause and send Kamal the shared-change request if:

- The publish action requires a DB write path not currently in `SihBrowserDal` or the Worker (R1 escalation)
- Any authoring field cannot be represented in the existing `OpportunityVersion` / `OpportunityRequirement` types
- Any recruiter view needs a field not in the `ProductionRecruiterProjection` allowlist